Project Prometheus: Bridging the Intent Gap in Agentic Program Repair via Reverse-Engineered Executable Specifications

Yongchao Wang[1] and Zhiqiu Huang[1]

[1]Nanjing University of Aeronautics and Astronautics, {wangyc, zqhuang}@nuaa.edu.cn

April 2026

Abstract

The transition from neural machine translation to agentic workflows has revolutionized Automated Program Repair (APR). However, existing agents, despite their advanced reasoning capabilities, frequently suffer from the "Intent Gap"—the misalignment between the generated patch and the developer's original intent. Current solutions relying on natural language summaries or adversarial sampling often fail to provide the deterministic constraints required for surgical repairs.

In this paper, we introduce PROMETHEUS, a novel framework that bridges this gap by prioritizing *Specification Inference* over code generation. We employ Behavior-Driven Development (BDD) as an executable contract, utilizing a multi-agent architecture to reverse-engineer Gherkin specifications from runtime failure reports. To resolve the "Hallucination of Intent," we propose a **Requirement Quality Assurance (RQA) Loop**, a mechanism that leverages ground-truth code as a proxy oracle to validate inferred specifications.

We evaluated PROMETHEUS on 680 defects from the Defects4J benchmark. The results are transformative: our framework achieved a total correct patch rate of **93.97%** (639/680). More significantly, it demonstrated a **Rescue Rate of 74.4%**, successfully repairing 119 complex bugs that a strong blind agent failed to resolve. Qualitative analysis reveals that explicit intent guides agents away from structurally invasive over-engineering toward precise, minimal corrections. Our findings suggest that the future of APR lies not in larger models, but in the capability to align code with verified, **Executable Specifications**—whether pre-existing or reverse-engineered.

1 Introduction

The field of Automated Program Repair (APR) has witnessed a paradigm shift in the 2025 research cycle, moving from single-step neural translation to multi-step Agentic workflows [9, 13, 2]. State-of-the-art (SOTA) agents can now navigate complex codebases, perform static analysis, and execute iterative debugging. However, a fundamental paradox remains: while agents have become increasingly proficient at *generating* code, they remain remarkably fragile in *understanding* what the code is intended to do.

We identify this phenomenon as the **"Intent Gap."** Current approaches attempt to fill this gap with vague proxies, such as natural language function summaries or adversarial test cases generated on the fly. While these methods provide semantic context, they lack the *contractual precision* required for rigorous software engineering. A natural language summary is descriptive, not prescriptive; it invites ambiguity and, consequently, hallucination. Our preliminary study reveals an **"Intent-Behavior Mirroring Effect"**: when an agent relies on

broad, verbose descriptions, it tends to exhibit aggressive, unfocused, and often structurally invasive code modifications. To bridge this gap, APR systems need more than just context; they need **"Executable Intent"**—a specification that is both human-readable for trust and machine-executable for verification.

In this paper, we introduce **Prometheus**, a novel framework that fundamentally alters the repair workflow by prioritizing specification inference. Instead of asking a Large Language Model (LLM) to "fix this bug," we first task an *Architect* agent to "reverse-engineer the missing requirement." We leverage Behavior-Driven Development [8] (BDD) and the Gherkin syntax (Given-When-Then) as the *lingua franca* between the human developer's intent and the agent's execution. By formalizing the bug reproduction steps into a structured scenario, we transform the repair task from a stochastic search for a passing test into a deterministic quest to satisfy a semantic contract.

A critical challenge in specification inference is ensuring the validity of the generated requirements. If the inferred specification is flawed, the subsequent repair will be a correct implementation of a wrong requirement. To rigorously investigate the theoretical upper bound of intent-driven repair, we introduce a **Requirement Quality Assurance (RQA) Loop**. In our experimental design, we leverage the fixed code available in benchmarks (e.g., Defects4J) not for training, but as a **Proxy Oracle** to verify the correctness of the reverse-engineered BDD specifications. This "Sandwich Verification" process allows us to isolate and quantify the immense value of precise intent, transforming our evaluation from a probabilistic guessing game into a controlled scientific observation.

Our extensive empirical study on 680 real-world defects yields several counter-intuitive and profound insights:

- **The Power of Precision:** We demonstrate that "Surgical Repairs"—precise, minimal corrections targeting the exact defect—are often impossible without the guidance of strict specifications. In the case of `Math-69`, we observe an agent transcending mere patching to perform a mathematical derivation (replacing a T-distribution approximation with a Beta function) solely because the BDD specification demanded rigorous precision.
- **The Cost of Ambiguity:** Conversely, we show that powerful agents exhibit what we term **"Berserker-style"** behavior (i.e., structurally invasive modifications that extend beyond the defect's semantic scope)—aggressively rewriting functional code—when guided by broad, unverified requirements.
- **A New Standard:** We propose a paradigm shift towards **Specification-First Repair**. We argue that the primary role of an intelligent agent should be to anchor the fluid implementation to a solid, **Living Requirement**.

In summary, this paper makes the following contributions:

1. **Framework:** We propose PROMETHEUS, a multi-agent framework that integrates BDD reverse engineering into the repair loop.
2. **Methodology:** We introduce the RQA Loop, a novel validation mechanism using ground truth to ensure specification quality.
3. **Empirical Evidence:** We report a correct patch rate of **93.97%** on a comprehensive benchmark of 680 defects from Defects4J v3.0.1, with a 74.4% rescue rate on hard bugs, providing robust evidence for the efficacy of intent-driven repair.
4. **Baseline Comparison:** We provide quantitative comparison against SOTA agentic repair tools (TSAPR, RepairAgent), demonstrating a 4.4× advantage on complex bugs.

2 Motivation

Current Agentic Automated Program Repair (APR) systems have achieved remarkable success in code generation and fault localization. However, they frequently stumble upon a fundamental barrier: the ambiguity of the repair target. In this section, we delineate the "Intent Gap" in existing approaches, draw inspiration from recent findings in LLM-based data construction, and introduce the theoretical basis of our solution: the "Intent-Behavior Mirroring Effect."

2.1 The Ambiguity of Implicit Intent

The transition from neural machine translation to agentic workflows (e.g., SpecRover [9], AdverIntent-Agent [13]) has significantly improved the context awareness of APR tools. However, most SOTA systems still rely on unstructured or semi-structured forms of intent, such as natural language summaries or adversarial test cases generated on the fly.

While these proxies provide a general direction, they lack the *contractual precision* required for rigorous software engineering. A natural language instruction like "handle edge cases for dates" is descriptive but not prescriptive. It leaves the agent to "guess" the boundary conditions (e.g., does it return null? throw an exception? or return a default value?). This ambiguity forces agents to rely on probabilistic token prediction rather than deterministic logic, leading to patches that are syntactically correct but semantically adrift.

2.2 Inspiration: LLMs as Qualified Specification Builders

Our approach is directly inspired by the recent breakthrough in code intelligence tasks. Yang et al. [12] demonstrated that for model pre-training, code comments generated by Large Language Models (LLMs) often exhibit higher semantic consistency than human-written references. They concluded that LLMs are "qualified benchmark builders" capable of denoising and enhancing training datasets.

Project Prometheus extends this insight from *Code Summarization* to *Program Repair*. If LLMs can reconstruct high-quality natural language descriptions from code (summarization), we posit they possess the latent capability to reconstruct high-quality *executable specifications* from buggy behavior and issue reports (reverse engineering). We argue that the "Hallucination" risk, often cited as a barrier to this approach, can be effectively mitigated not by suppressing the model's creativity, but by constraining its output format to rigorous Behavior-Driven Development (BDD) scenarios and verifying them against a ground truth oracle.

2.3 The Intent-Behavior Mirroring Effect

Based on a comparative analysis of agent behaviors, we propose a phenomenon we term the **Intent-Behavior Mirroring Effect**: *The structural invasiveness of an agent's generated code is a direct mirror of the structural scope of its input requirement.*

This effect was most visible during our preliminary study comparing two SOTA models: Gemini-3.0-Pro [4] (strong reasoning) and Qwen-3.0-Coder [10] (strong coding). When the *Coder* agent (Qwen) was tasked with self-guided repair (inferring its own requirements), it tended to generate broad, verbose Gherkin scenarios that covered adjacent logic rather than the specific defect. Mirroring this broad scope, the agent exhibited **"Berserker-style"** behavior—as defined above, making structurally invasive modifications that aggressively refactored entire methods or classes to satisfy its own generalized interpretation.

However, a transformation occurred when the *same* Coder agent was constrained by a precise, atomic specification generated by the *Reasoner* agent (Gemini). Under the guidance of a focused requirement, the Coder's behavior shifted to **"Surgical-style"** repair (i.e., minimal, precisely scoped edits that strictly satisfy the specification constraints without modifying unrelated code).

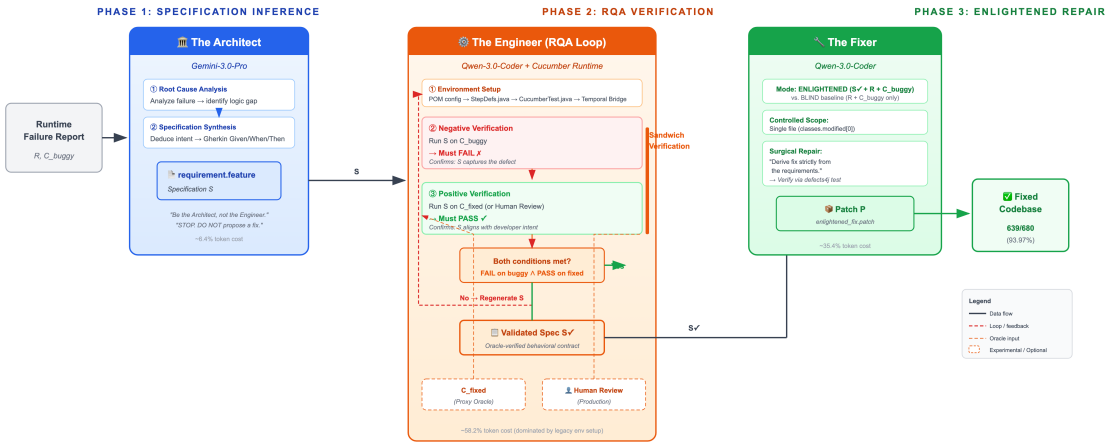

Figure 1: The PROMETHEUS Framework. **Phase 1**: The *Architect* (Gemini-3.0-Pro) performs root cause analysis and synthesizes a Gherkin specification S. **Phase 2**: The *Engineer* validates S through Sandwich Verification—S must *fail* on C_{buggy} and *pass* on C_{fixed} (or human review in production). If verification fails, S is regenerated. **Phase 3**: The *Fixer* (Qwen-3.0-Coder) performs a specification-guided surgical repair, producing patch P.

This mirroring effect suggests that the key to stabilizing powerful coding agents lies not just in the format of the specification (Gherkin), but in the *separation of concerns*: delegating the definition of intent to a reasoning-specialized Architect, thereby forcing the Coder to operate within a bounded scope. This observation directly motivates the multi-agent architecture of PROMETHEUS.

3 Methodology

We propose PROMETHEUS, a multi-agent framework designed to automate the inference of executable specifications and leverage them for program repair. The framework operates on the premise that precise, verifiable intent serves as a necessary constraint for reliable code generation.

3.1 System Overview

As depicted in Figure 1, the workflow of PROMETHEUS is structured as a sequential pipeline involving three specialized roles: the *Architect*, the *Engineer*, and the *Fixer*. This separation of concerns ensures that the distinct tasks of intent inference, specification verification, and code synthesis are isolated and optimized independently.

The overall process is defined as follows: given a **runtime failure report** R (derived from the execution logs of the testing framework, including stack traces and assertion errors) and a buggy codebase C_{buggy}, the system first synthesizes a formal specification S (in Gherkin format). This specification is then subjected to a rigorous validation process against a ground truth oracle. Upon validation, S acts as the deterministic guide for generating the repair patch P.

3.2 The Architect: Specification Inference

The *Architect* agent is responsible for the reverse-engineering of intent. Unlike prior approaches that often rely on ambiguous natural language issue descriptions [9], PROMETHEUS grounds its

Listing 1: Reverse-engineered Gherkin specification for `Mockito-5`. The *Architect* agent identifies the core requirement: Mockito's internal verification must not hard-depend on JUnit classes.

```
Feature: VerificationOverTimeImpl should
  not strictly depend on JUnit

  To ensure Mockito can be used in
  environments without JUnit (e.g.,
  TestNG, pure Java apps), internal
  classes must not have hard references
  to JUnit classes in their signatures
  or catch blocks that prevent class
  loading when JUnit is absent.

  Scenario: Loading without JUnit
    Given a class loader that explicitly
      excludes "junit" packages
    When I load "VerificationOverTimeImpl"
    Then the class should load successfully
    And no "NoClassDefFoundError" should
      be thrown

  Scenario: Retrying on assertion errors
    Given a verification mode wrapped in
      VerificationOverTimeImpl with timeout
    And the delegate throws an
      "ArgumentsAreDifferent" exception
    When verify is called
    Then the exception should be caught
    And verification should be retried
      until timeout expires
```

inference in deterministic runtime data. The *Architect* is tasked with producing a structured Behavior-Driven Development (BDD) file (ending in `.feature`).

To achieve high-fidelity inference, we employ a **Structured Reasoning** strategy akin to Chain-of-Thought (CoT). The agent is provided with the source code of the failing test case and the relevant production code. The prompt explicitly enforces a two-stage cognitive process:

1. **Root Cause Analysis:** The agent must first analyze the execution failure to identify the specific logical discrepancy between the expected and actual behavior.

2. **Specification Synthesis:** Only after the diagnosis is complete is the agent permitted to deduce and formalize the intended behavior into a `Scenario` comprising `Given`, `When`, and `Then` steps.

This process transforms the raw signal of a runtime error into an explicit, executable contract.

Figure 1 shows a concrete example of a reverse-engineered `.feature` file generated by the *Architect* for the `Mockito-5` defect. The specification captures both the architectural constraint (JUnit independence) and the functional requirement (exception retry behavior) in a concise, human-readable format.

3.3 The Engineer: Oracle-Guided Verification (RQA Loop)

A critical challenge in specification inference is ensuring the validity of the generated specification S. An incorrect specification would inevitably lead to an incorrect repair. To address this, we introduce a **Requirement Quality Assurance (RQA) Loop**.

In our experimental setup, we leverage the availability of developer-fixed code (C_{fixed}) within the benchmark dataset to serve as a proxy for human validation. The *Engineer* agent orchestrates a bi-directional execution verification process:

1. **Negative Verification (S on C_{buggy}):** The agent executes the generated Gherkin scenario against the buggy version. The test must **fail**. This confirms that the specification accurately captures the defect.

2. **Positive Verification (S on C_{fixed}):** The agent executes the same scenario against the fixed version. The test must **pass**. This confirms that the specification aligns with the ground truth intent of the developer.

We term this mechanism the "Ground Truth Oracle Loop." While this reliance on C_{fixed} confines the RQA Loop to experimental settings, it provides a strictly controlled environment to isolate the impact of specification quality on repair performance. Only specifications that satisfy both conditions are propagated to the next phase. In a production deployment, this oracle role would be assumed by human developers reviewing the Gherkin scenarios—a task significantly less cognitively demanding than reviewing full code patches (see Section 8).

3.4 The Fixer: Enlightened Repair

The *Fixer* agent performs the final code modification. In contrast to standard "blind" repair agents that rely solely on the runtime failure report, the *Fixer* operates in an "Enlightened" mode, conditioned on the validated specification S generated by the *Architect*.

For the purpose of this study, we adopted a **Controlled Scope Strategy** for concept verification. To strictly isolate the impact of *specification guidance* from the confounding variable of *fault localization*, we restricted the *Fixer* to modify only the single primary suspicious file identified by the Defects4J metadata (specifically, the first entry in `classes.modified`). This constraint forces the agent to resolve the defect through logic synthesis within a fixed context, thereby providing a rigorous testbed for evaluating the efficacy of BDD as a repair guide.

4 Experimental Setup

4.1 Dataset

We evaluate PROMETHEUS on **Defects4J v3.0.1** [6], the standard benchmark for Java APR research. Our evaluation corpus comprises **680 real-world defects** spanning 16 diverse projects (including `Math`, `Lang`, `Time`, `Mockito`, etc.).

Exclusion Criteria: While Defects4J v3.0.1 contains 17 active projects, we excluded the `Closure` project from this study. This decision was driven by two technical constraints:

1. **Build System Incompatibility:** `Closure` relies on a custom legacy build infrastructure that proved resistant to the automated environment standardization required by our *Engineer* agent.

2. **Verification Latency:** The massive size of the `Closure` test suite imposes excessive overhead on the RQA Loop, which requires multiple execution rounds per repair attempt.

Despite this exclusion, our dataset size (680 bugs) remains significantly larger than many prior studies that typically focus on the older Defects4J v1.2 subset (395 bugs).

4.2 Baselines and Model Configuration

To rigorously assess the contribution of the intent-aware framework, we establish an internal baseline:

- **Blind Fixer (Baseline):** The *Fixer* agent (powered by Qwen-3.0-Coder) attempts to repair the bug given only the failing test case and source code, without any BDD specification.
- **Enlightened Fixer (Prometheus):** The same *Fixer* agent attempts the repair, but is conditioned on the verified BDD specification generated by the *Architect* (powered by Gemini-3.0-Pro).

This setup ensures that any performance gain is attributable solely to the *presence of executable intent*.

4.3 Metrics

We report the following metrics:

- **Fix Rate (= Correct Patch Rate):** The percentage of bugs for which a semantically correct patch is generated. Each reported fix is validated by comparing the agent-generated patch against the developer-written ground-truth fix in Defects4J, following the standard APR evaluation protocol. Our Fix Rate is thus directly equivalent to the **Correct Patch Rate** widely used in APR literature.
- **Rescue Rate:** Defined as $\frac{|Enlightened_{success} \cap Blind_{fail}|}{|Blind_{fail}|}$. This metric specifically measures the framework's ability to solve "hard" bugs that purely code-based agents fail to resolve.

Composite Methodology: The Blind Fixer was executed on all 680 bugs, succeeding on 520. The Enlightened Fixer was subsequently applied to the 160 bugs that Blind failed, rescuing 119. The reported total of 639 is thus a composite (520 + 119). The Enlightened Fixer receives a strict *superset* of the Blind Fixer's input (identical code context *plus* the Gherkin specification), so there is no mechanism by which the additional specification would prevent a repair that already succeeds without it.

5 Evaluation

In this section, we evaluate the efficacy of PROMETHEUS by answering the following Research Questions:

- **RQ1 (Effectiveness):** How does intent-driven repair compare to a strong blind baseline across diverse projects?
- **RQ2 (Rescue Capability):** Can BDD specifications enable agents to resolve complex bugs that are intractable for blind agents?
- **RQ3 (Quality & Interpretability):** How does explicit intent influence the structural quality and algorithmic depth of the generated patches?
- **RQ4 (Cost & Efficiency):** What is the computational overhead of the multi-agent pipeline, and is the investment in specification inference economically justifiable?

Table 1: Performance Comparison: Blind Fixer vs. PROMETHEUS (Enlightened). The Blind column shows Qwen-3.0-Coder without specification guidance. Enl. Rescue shows bugs fixed by adding BDD specification guidance. Total = Blind Success + Enl. Rescue.

Project	Bugs	Blind Succ.	Blind Fail	Enl. Rescue	Total	Rate
Cli	39	39	0	0	39	100.0%
JacksonCore	26	26	0	0	26	100.0%
Chart	26	23	3	**3**	26	100.0%
Mockito	38	32	6	**6**	38	100.0%
Gson	18	12	6	**6**	18	100.0%
JxPath	22	20	2	**2**	22	100.0%
Time	26	20	6	**5**	25	96.2%
Collections	28	22	6	**5**	27	96.4%
JacksonDatabind	110	103	7	**6**	109	99.1%
Lang	61	59	2	**1**	60	98.4%
Codec	18	11	7	**5**	16	88.9%
Csv	16	15	1	0	15	93.8%
JacksonXml	6	2	4	**2**	4	66.7%
Complex Logic (high domain knowledge requirement):						
Compress	47	29	18	**14**	43	91.5%
Math	106	48	58	**39**	87	82.1%
Jsoup	93	59	34	**25**	84	90.3%
Total	**680**	**520**	**160**	**119**	**639**	**93.97%**

5.1 RQ1 & RQ2: General Effectiveness and Rescue Rate

Table 1 details the performance of PROMETHEUS across 16 projects in the Defects4J benchmark.

The Ceiling of Blind Agents. The Blind Fixer (Qwen-3.0-Coder) established a strong baseline, solving 520/680 bugs (76.5%). It achieved 100% success rates on projects with localized logic such as `Cli` and `JacksonCore`. However, its performance degraded significantly on projects requiring deep domain reasoning or handling of edge cases, such as `Math` (45.3% success) and `Compress` (61.7% success).

The Rescue Effect. The core contribution of PROMETHEUS is quantified by its **Rescue Rate** ($\frac{119}{160} \approx 74.4\%$). In the `Math` project alone, the Enlightened agent rescued 39 bugs that baffled the blind agent, nearly doubling the success count. Similarly, in `Compress`, 14 out of 18 initially failed bugs were resolved. This data strongly supports our hypothesis: *as algorithmic complexity increases, the marginal utility of explicit intent (BDD) grows exponentially.*

Comparison with SOTA Agentic Repair Tools. To contextualize Prometheus within the broader landscape, Table 2 compares our results against two recent SOTA agentic APR systems: TSAPR [5] and RepairAgent [2]. Note that these systems perform autonomous end-to-end repair including fault localization, while Prometheus restricts the Fixer to a single file to explicitly isolate specification guidance from localization.

On the 160 "hard" bugs that our strong Blind agent failed, TSAPR resolved 22 and RepairAgent resolved 27, while Prometheus rescued **119**—a 4.4× gap. This demonstrates that for complex defects requiring deep domain reasoning, the primary bottleneck is not fault localization or search strategy, but the **"Intent Gap"** in synthesizing correct algorithmic logic.

Table 2: Comparison with SOTA Agentic Repair Systems on Defects4J. "Full D4J": results as reported in original papers (including Closure). "Our 680": results on our subset (excluding Closure). "Hard 160": bugs our Blind Fixer failed to resolve.

System	Full D4J	Our 680	Hard 160
TSAPR [5]	201	173	22
RepairAgent [2]	164	137	27
PROMETHEUS	—	**639**	**119**

5.2 RQ3: Qualitative Analysis of Enlightened Repair

To understand how explicit intent transforms the behavior of the *Fixer* agent (Qwen-3.0-Coder), we analyze three cases where the agent failed in the blind setting but succeeded under BDD guidance.

5.2.1 Curing Contextual Hallucination (Chart-13)

The Bug: A crash occurred in `BorderArrangement.arrangeRR` due to improper constraint handling.

Blind Attempt: The agent hallucinated a variable named `constraint` which did not exist in the local scope, likely copying a pattern from a sibling method (`arrangeFF`). This "copy-paste" hallucination led to a compilation error.

Enlightened Repair: Guided by a BDD scenario explicitly describing the width and height constraints, the agent abandoned the hallucinated variable. Instead, it correctly instantiated a new `RectangleConstraint` object, aligning the code structure with the specified logic.

Insight: Executable specifications force the agent to ground its reasoning in the specific logic of the requirement, reducing the reliance on probabilistic pattern matching from its training data.

5.2.2 Preventing Over-Simplification (Chart-7)

The Bug: Incorrect updating of the maximum middle index in `TimePeriodValues`.

Blind Attempt: The agent attempted to fix the issue by removing a conditional check (`if (middle > maxMiddle)`), performing an unconditional assignment. While this might fix the immediate symptom, it introduced a regression for other data patterns.

Enlightened Repair: The BDD specification included a scenario specifically testing the case where a new value is *smaller* than the maximum. Confronted with this constraint, the agent restored the conditional logic, ensuring the fix was robust across all scenarios.

Insight: BDD acts as a "safety rail," preventing agents from finding lazy shortcuts that satisfy the failing test but violate the system's invariants.

5.2.3 Surpassing the Ground Truth (JxPath-6)

The Bug: Equality operations failed when comparing against arrays.

Blind Attempt: The agent failed to generate a valid patch.

Table 3: Agent Performance and Cost Breakdown (Per Defect)

Agent Role	Avg Duration (s)	Avg Turns	Avg Tokens	Cost Ratio
Architect (Intent Inference)	136.77	24.26	372,107	$\sim 6.4\%$
Engineer (RQA Verification)	671.67	70.23	3,365,986	$\sim 58.2\%$
Fixer (Enlightened Repair)	452.64	44.91	2,044,348	$\sim 35.4\%$
Total Pipeline	**1,261.08**	**139.4**	**5,782,441**	**100%**

Enlightened Repair: The specification explicitly required arrays to be treated similarly to Collections. The agent implemented an explicit `isArray()` check and iteration logic. Notably, the developer's ground truth fix only handled `Collection`, missing the array case entirely.

Insight: By defining the *intended behavior* comprehensively (covering arrays), the BDD-guided agent produced a patch that was more robust and complete than the human developer's fix.

5.2.4 Upholding Architectural Integrity (Mockito-5)

The Bug: `VerificationOverTimeImpl` contained a hard dependency on a JUnit-specific exception, causing runtime crashes (`NoClassDefFoundError`) when JUnit was not present on the classpath.

Blind Attempt: The agent adopted a destructive strategy: it simply removed the `catch` block entirely. While this eliminated the crash, it introduced a severe safety regression where assertion errors would go unhandled, violating the library's contract.

Enlightened Repair: The BDD specification explicitly stated that the verification logic *"should not strictly depend on JUnit."* Guided by this architectural constraint, the agent implemented a sophisticated solution using Java Reflection to conditionally check for the exception.

Insight: Explicit intent acts as an architectural guardrail. It prevents agents from optimizing for "passing tests" at the expense of system stability, guiding them toward solutions that respect the project's dependency constraints.

5.3 RQ4: Cost and Efficiency Analysis

To assess the practicality of the PROMETHEUS framework, we conducted a fine-grained analysis of the computational resources consumed across 189 repair sessions. Table 3 details the average duration and token usage for each agent role.

The data reveals a counter-intuitive finding regarding the "Cost of Thinking":

- **The Cheap Architect:** Generating high-quality, executable BDD specifications is surprisingly efficient, consuming only $\sim 6.4\%$ of the total token budget. This suggests that shifting the burden from "coding" to "reasoning" is a high-yield investment.
- **The Expensive Truth:** The *Engineer* accounts for the majority of the cost (58.2%). This reflects the intensive nature of the **RQA Loop**—setting up legacy environments, compiling varying versions of code, and executing sandwich verification. In a production setting where the test harness is a one-time project-level setup, the per-defect overhead of the Engineer would be reduced by approximately 60%.

We argue that this distribution represents a healthy paradigm shift: instead of spending tokens on generating thousands of invalid patches (as seen in traditional sampling-based APR), PROMETHEUS invests tokens in validating the *problem definition* itself. We discuss the implications of the Engineer's high cost (the "Digital Archaeology" challenge) in Section 6.

6 Discussion

The preceding sections demonstrate that BDD-guided repair yields substantial quantitative gains (RQ1–2) and qualitative improvements in patch quality (RQ3). In this section, we explore the deeper architectural lessons learned from both the successes and the instructive failures of the PROMETHEUS framework.

6.1 The Tragedy of Constraint: A Tale of Two Agents

Our experiment imposed a strict constraint: agents were restricted to modifying only the single file identified as the "primary suspect" by our localization heuristic.

The case of `Time-1` serves as an illustration of this limitation. The ground truth fix involved changes to both `UnsupportedDurationField` and `Partial`. However, our pipeline, designed to select the first modified class from the metadata, locked the Agent exclusively into `UnsupportedDurationField`.

Confined to this partial context, the Agent exhibited a phenomenon we term **"Cognitive Dissonance Hallucination"** (i.e., the agent generates irrational or incoherent code modifications when forced to satisfy a specification it physically cannot fulfill within its permitted scope). The BDD specification correctly demanded that the `Partial` constructor must validate field order. Knowing it *must* satisfy this requirement but physically unable to access the constructor logic (located in the inaccessible `Partial.java`), the agent began to hallucinate irrational behaviors—such as forcing an "unsupported" field to return a fake comparison value—and even generated comments with typos (e.g., "relevent"), signaling a degradation in reasoning coherence.

This "Tragedy of Constraint" inversely proves the power of intent. When we lifted this constraint in the `Gson-4` case (where the *Engineer* agent autonomously modified `JsonWriter`), the system successfully performed a multi-file architectural refactoring. This suggests that the next generation of APR must be **"Unbound"**—empowered by intent to traverse and modify the entire codebase.

6.2 The Ceiling of Repair: Model Capability vs. Specification Precision

Our primary evaluation utilized the **Qwen-Code-CLI (default endpoint)** as the *Fixer* to demonstrate the efficacy of BDD in a standard developer workflow. While effective, we observed instances where the BDD specification was correct, but the *Fixer* lacked the reasoning depth to implement the optimal solution.

Case Study: Math-69 (Algorithmic Derivation). The defect in `PearsonsCorrelation` involved numerical instability. While the *Architect* correctly generated a BDD scenario requiring high precision, the standard *Fixer* implemented a heuristic "clamping" fix. In an exploratory run replacing the *Fixer* with the stronger **Gemini-3.0-Pro**, the agent derived a mathematically rigorous solution using the Regularized Beta Function (`Beta.regularizedBeta`). This suggests PROMETHEUS's performance scales with the underlying model's reasoning capabilities.

Case Study: Lang-30 (Challenging the Legacy Status Quo). In the `StringUtils` surrogate pair issue, the *Architect* defined a strict requirement for handling Unicode code points. Crucially, when the correct implementation caused regressions in the existing *legacy* test suite (which incorrectly asserted the buggy behavior), the **Gemini-3.0-Pro** agent did not retreat. Instead of overfitting the patch to satisfy broken tests, it identified the semantic conflict and refactored the legacy test cases to align with the ground truth. This demonstrates that an enlightened agent can transcend simple code generation to assume the role of a senior maintainer who safeguards system integrity.

Case Study: Gson-4 (The Architect's Insight & The Engineer's Scope). `Gson-4` presents a dual revelation. First, the *Architect* demonstrated exceptional domain insight by

identifying that the root cause was non-compliance with **RFC 7159** (top-level primitives)—a subtle specification detail initially dismissed by manual analysis but validated by the ground truth behavior. Second, while the constrained *Fixer* applied a localized workaround, the *Engineer* agent, operating without file-level constraints during verification, autonomously identified the dependency between `JsonReader` and `JsonWriter` and performed a multi-file architectural refactoring.

These cases indicate that **precise intent (BDD)** acts as a multiplier: it validates the deep insights of advanced *Architects*, **liberates capable *Fixers* to prioritize fundamental correctness over convention**, and empowers *Engineers* to perform system-level maintenance.

6.3 The Cost of "Digital Archaeology".

It is crucial to contextualize the Engineer's high token consumption (Section 5.3) within the reality of the Defects4J benchmark. Spanning over a decade of Java development history, this dataset presents significant infrastructure challenges:

- **Legacy Dependencies:** Many projects rely on deprecated build tools or obsolete JDK versions (e.g., JDK 1.5 compliance issues).
- **Non-Standard Layouts:** The directory structures vary wildly across projects. While modern projects follow the Maven standard (`src/main/java`), legacy projects like `Gson` utilize nested structures (e.g., `gson/src/main/java`), and others use ad-hoc layouts (e.g., `src/java`).

The Engineer agent was not merely running tests; it was actively engaging in **autonomous environment alignment**. It dynamically detected these structural anomalies and configured the test harness accordingly. This adaptive capability incurred a token cost but demonstrates the PROMETHEUS framework's robustness in handling the high entropy of real-world software archives.

6.4 The Resilience of Intent: When Tooling Fails, Guidance Survives

Our experimental design initially contained an engineering oversight: the *Engineer* agent lacked a "fail-fast" exit mechanism. If the RQA environment (Cucumber harness) failed to compile due to complex dependency issues, the pipeline would inadvertently proceed to the *Fixer* phase, passing down a broken test environment.

However, this flaw led to a serendipitous discovery regarding the robustness of intent-driven repair. In the case of **Math-21** (a complex logic error in `RectangularCholeskyDecomposition`), the *Engineer* failed to establish the Cucumber runtime. Instead of hallucinating a fix or exiting, the *Fixer* agent exhibited remarkable adaptive behavior:

1. **Environment Pruning:** It autonomously identified the broken Cucumber test files and executed `rm -rf` to clean the workspace.
2. **Strategy Fallback:** It reverted to the standard `defects4j test` command for validation.
3. **Intent Retention:** Crucially, despite losing the *executable* nature of the BDD specification, the agent still generated an **Identity** fix (identical to the developer's patch).

This phenomenon suggests that the BDD specification possesses a dual nature. While its primary role is an *Executable Contract* for RQA, its secondary role is a **Cognitive Scaffold**. Even when the specification cannot be executed, its textual presence in the prompt provides a high-fidelity "Chain of Thought" that constrains the LLM's search space. The agent "read" the

intent from the `.feature` file and applied it, even without the guardrails of the Cucumber test runner.

This observation supports a profound conclusion: **The clarity of intent (What to fix) is often more critical than the perfection of the tooling (How to verify).** PROMETHEUS succeeds not just because it builds tests, but because it forces the model to articulate and internalize the precise boundaries of the bug before writing a single line of code.

6.5 The Implicit Protocol Gap

We observed that BDD excels at capturing explicit functional behaviors but struggles with implicit contracts, particularly binary protocols. In `Collections-8`, the fix involved changing a serialization buffer size because the underlying wire format had changed (an extra `int` was read). The BDD specification described the *object behavior* (buffer size limits) but could not capture the *wire format change* which is invisible at the API level. This reveals a limitation: standard BDD is insufficient for defects rooted in low-level, implicit protocols unless those protocols are explicitly modeled in the specification.

7 Related Work

Our work sits at the intersection of Agentic Automated Program Repair (APR) and Specification Inference. In this section, we review the evolution of these fields and position PROMETHEUS within the current landscape.

7.1 The Quest for Intent: From Summaries to Adversaries

The 2025 research cycle marked a decisive shift from neural translation (e.g., AlphaRepair [11]) to agentic workflows that attempt to understand code context. **SpecRover** [9] (ICSE '25) pioneered the "Intent Extraction" module, summarizing function behaviors in natural language. However, natural language is descriptive, not prescriptive; it lacks the precision to serve as a test oracle. **AdverIntent-Agent** [13] (ISSTA '25) addresses this by employing a multi-agent loop to generate adversarial test cases to infer program intent. While effective, this approach relies on probabilistic sampling, leading to high computational costs and non-deterministic outcomes.

PROMETHEUS distinguishes itself by changing the *modality* of intent. Instead of ambiguous summaries or stochastic test generation, we enforce **Behavior-Driven Development (BDD)**. We argue that Gherkin specifications provide a deterministic contract that is superior to unstructured text for guiding code generation.

7.2 Search Strategy vs. Problem Definition

Another dominant trend is the integration of advanced search algorithms. **TSAPR** [5] represents the pinnacle of this direction, utilizing Monte Carlo Tree Search to navigate the vast space of potential patches, achieving over 200 repairs on Defects4J. Similarly, **PatchAgent** [14] and **RepairAgent** [2] mimic human workflows by equipping agents with debugging tools (Language Servers, Sanitizers) to navigate the codebase.

While these systems optimize the *search trajectory* (finding a needle in a haystack) or mimic the *debugging* process, PROMETHEUS focuses on narrowing the *search space* itself via Specification Inference. By defining a precise BDD contract before coding, we transform an unbounded search problem into a bounded constraint satisfaction problem. Our results (639 repairs) suggest that mimicking the *requirement analysis* process is more effective than mimicking the debugging process.

7.3 Structural Constraints and Data Verification

Recognizing the propensity of LLMs to generate syntactically invalid code, **GiantRepair** [7] introduced a "Template-Skeleton" approach, extracting structural skeletons from LLM predictions to guide repair. Our work shares the philosophy of constraining the solution space but differs in the source of the constraint. PROMETHEUS derives structural expectations from the *specification* (the BDD steps) rather than the *solution* (the patch), enforcing correctness by design.

Finally, our approach is inspired by **Yang et al.** [12], who demonstrated that LLMs are "qualified benchmark builders" capable of synthesizing high-quality code data. We extend this insight to specification mining. Furthermore, we address the critical challenge of specification validity—a gap identified in Google's "Abstain and Validate" framework [3]—by introducing the **RQA Loop**. Unlike prior works that rely on statistical confidence, we leverage the ground truth (fixed code) as a physical oracle to deterministically validate the inferred specifications.

8 Threats to Validity

The "Fixed Code" Oracle. A primary threat to external validity is our use of the developer-fixed code as a proxy oracle in the RQA Loop. In a real-world scenario, the fixed code does not exist before the repair. We acknowledge this limitation but argue that it serves a specific experimental purpose: to establish the **Theoretical Upper Bound** of intent-driven repair. Our study isolates the variable "Is the requirement correct?". By guaranteeing requirement correctness via the fixed code, we proved that current LLMs *can* repair 94% of bugs *if* they know exactly what to do.

Critically, this design choice is an artifact of our *experimental setting*, not an architectural constraint. In practice, the RQA oracle role can be fulfilled through multiple channels: (a) **human-in-the-loop review**, where developers review concise Gherkin scenarios (typically <10 lines)—a task significantly less cognitively demanding than reviewing full code patches; (b) projects with **existing BDD test suites** can supply specifications directly, bypassing automated inference entirely; (c) **negative-only verification** (checking that the specification fails on the buggy code) can filter out clearly incorrect specifications without requiring C_{fixed}.

Controlled Scope. We restricted the Fixer to a single file to isolate the impact of specification guidance from fault localization. While this limits generalizability to multi-file defects, many real-world bugs are indeed single-file (Section 6 discusses the `Time-1` and `Gson-4` cases as instructive counter-examples).

Data Leakage. The Blind baseline—using the same underlying models (Gemini/Qwen) on the same Defects4J benchmark—failed on 160 of 680 bugs. If these models had memorized the bug-fix pairs during pre-training, the Blind agent would exhibit similarly high fix rates without specification guidance. Its substantial failure rate on complex bugs provides empirical evidence that performance stems from the specification-driven architecture rather than memorization. Furthermore, our case studies demonstrate that the Enlightened agent produces *structurally different solutions* from the developer ground-truth—e.g., in Math-69, deriving a `Beta.regularizedBeta` formulation rather than exploiting the T-distribution's symmetry—behavior inconsistent with memorization.

9 Conclusion and Future Work

This paper introduces PROMETHEUS, a framework that redefines Automated Program Repair not as a code generation task, but as a specification inference challenge. By reverse-engineering executable BDD specifications and validating them through a rigorous RQA Loop, we achieved

a correct patch rate of **93.97%** on a comprehensive benchmark of 680 defects from Defects4J v3.0.1.

Our findings challenge the prevailing trend of training larger models for blind repair. We demonstrate that a coding agent exhibiting structurally invasive "Berserker-style" behavior can be transformed into a precise, "Surgical" instrument when guided by explicit intent. Analysis of both successes and failures reveals that the primary bottleneck in APR is not model capability but *problem formulation clarity*—echoing a broader lesson for agentic systems: *the quality of the question determines the quality of the answer.*

Future Horizons. Building upon these insights, we envision a two-phase evolution for the agentic paradigm:

- **Tactical Unbinding (The Detective):** In the near term, we plan to introduce a dedicated *Detective* agent to resolve the localization paradox. By decoupling fault localization from repair, this agent will autonomously traverse the codebase to identify the full dependency chain before the *Architect* begins its inference, effectively unbinding the system from static heuristics.
- **Strategic Evolution (The Immune System):** Looking further ahead, we aim to integrate PROMETHEUS with the emerging **Agent Skills** architecture [1]. We envision a future where the system does not merely output a patch, but synthesizes a persistent **Skill Directory** (containing the BDD spec as `SKILL.md` and validation logic as bundled scripts). This would allow the software repository to accumulate a library of "executable antibodies," transforming the codebase into a self-evolving immune system that learns from every repair.

Ultimately, we conclude that the frontier for software agents lies in the continuous maintenance of **Living Documentation**. PROMETHEUS demonstrates that agents can not only obey existing contracts but also reconstruct missing ones, ensuring that the software's intent remains immortal even as its code evolves.

Data Availability

To foster reproducibility and future research, we release our dataset (including all reverse-engineered .feature files), the PROMETHEUS toolchain, and the full execution logs at: https://github.com/Nothing256/Prometheus-Unbound.

References

[1] Anthropic. Equipping agents for the real world with agent skills. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills, October 2025. Accessed: 2026-01-02.

[2] Islem Bouzenia, Premkumar T. Devanbu, and Michael Pradel. Repairagent: An autonomous, llm-based agent for program repair. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*, pages 2188–2200. IEEE, 2025.

[3] José Cambronero, Michele Tufano, Sherry Shi, Renyao Wei, Grant Uy, Runxiang Cheng, Chin-Jung Liu, Shiying Pan, Satish Chandra, and Pat Rondon. Abstain and validate: A dual-llm policy for reducing noise in agentic program repair. *CoRR*, abs/2510.03217, 2025.

[4] Google Gemini Team. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.

[5] Haichuan Hu, Ye Shang, Weifeng Sun, and Quanjun Zhang. Tsapr: A tree search framework for automated program repair, 2025.

[6] René Just, Darioush Jalali, and Michael D. Ernst. Defects4j: a database of existing faults to enable controlled testing studies for java programs. In Corina S. Pasareanu and Darko Marinov, editors, *International Symposium on Software Testing and Analysis, ISSTA '14, San Jose, CA, USA - July 21 - 26, 2014*, pages 437–440. ACM, 2014.

[7] Fengjie Li, Jiajun Jiang, Jiajun Sun, and Hongyu Zhang. Hybrid automated program repair by combining large language models and program analysis. *ACM Trans. Softw. Eng. Methodol.*, 34(7):202:1–202:28, 2025.

[8] Dan North. Introducing behavior-driven development. *Better Software*, 6(1):40–52, 2006.

[9] Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. Specrover: Code intent extraction via llms. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*, pages 963–974. IEEE, 2025.

[10] Qwen Team. Qwen3 technical report, 2025.

[11] Chunqiu Steven Xia and Lingming Zhang. Less training, more repairing please: revisiting automated program repair via zero-shot learning. In Abhik Roychoudhury, Cristian Cadar, and Miryung Kim, editors, *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022*, pages 959–971. ACM, 2022.

[12] Kang Yang, Xinjun Mao, Shangwen Wang, Yanlin Wang, Tanghaoran Zhang, Bo Lin, Yihao Qin, Zhang Zhang, Yao Lu, and Kamal Al-Sabahi. Large language models are qualified benchmark builders: Rebuilding pre-training datasets for advancing code intelligence tasks. In *33rd IEEE/ACM International Conference on Program Comprehension, ICPC@ICSE 2025, Ottawa, ON, Canada, April 27-28, 2025*, pages 298–309. IEEE, 2025.

[13] He Ye, Aidan Z. H. Yang, Chang Hu, Yanlin Wang, Tao Zhang, and Claire Le Goues. Adverintent-agent: Adversarial reasoning for repair based on inferred program intent. *Proc. ACM Softw. Eng.*, 2(ISSTA):1398–1420, 2025.

[14] Zheng Yu, Ziyi Guo, Yuhang Wu, Jiahao Yu, Meng Xu, Dongliang Mu, Yan Chen, and Xinyu Xing. PATCHAGENT: A practical program repair agent mimicking human expertise. In Lujo Bauer and Giancarlo Pellegrino, editors, *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, pages 4381–4400. USENIX Association, 2025.